

Guia de execução — P2 (acabamento documental, coerência narrativa e hardening leve)

Cap. 1 — Introdução e Contexto

P2.1 Fronteiras de sistema no Cap. 1

- **Por quê:** alinhar a introdução ao que já está formalizado (diagramas/contratos).
- **Como:** inserir **diagrama de contexto** LLM ↔ TER-Kai ↔ Usuário e apontar para `/api/v1/chat/invoke`, `/api/v1/logs`, `/api/v1/policies`.
- **Entregável:** `/docs/cap1_context.md` (1 página com diagrama Mermaid + links para OpenAPI).
- **Dono:** Eng + Compliance.
- **Aceitação:** diagrama bate com topologia (P0.6) e OpenAPI (P1.45) sem divergências.

P2.2 Catálogo de métricas externas

- **Por quê:** Cap. 1 declara dependência sem listar fontes/formato/validação.
- **Como:** tabela “fonte → formato → validação → hash de referência → periodicidade”.
- **Entregável:** `/docs/metrics_catalog.md`.
- **Dono:** Data/Eng.
- **Aceitação:** cada métrica usada no MIR tem **hash de referência** registrado no PoP-L.

P2.3 Glossário com versões/schemas

- **Por quê:** termos (Kai, MIR, Ω ...) sem IDs/versões.
- **Como:** ancorar cada termo a um **schema canônico** (do catálogo P1.7) e versão `contracts-vX`.
- **Entregável:** `/docs/glossario.md`.
- **Dono:** Compliance + Eng.
- **Aceitação:** todo termo do Cap. 1 aponta para um `$id` de schema.

P2.4 Premissas testáveis

- **Por quê:** ligar premissas do Cap. 1 às evidências (Cap. 6).
- **Como:** para cada premissa, referenciar **KPI + caso V&V (ID)** + link do PoP-L.
- **Entregável:** seção “Premissas & Testes” em `/docs/cap1_context.md`.
- **Dono:** QA + Compliance.
- **Aceitação:** nenhum enunciado sem **teste/ID/threshold** associado.

P2.5 Papéis/RBAC no Cap. 1

- **Por quê:** ancorar papéis e SoD na abertura.
 - **Como:** mini-matriz de papéis com link para `/api/v1/review-queue` e política SoD (P1.19).
 - **Entregável:** addenda no Cap. 1 (no mesmo arquivo do item 1).
 - **Dono:** Compliance.
 - **Aceitação:** papéis batem com enum `Role` migrada (P1.42).
-

Cap. 2 — Arquitetura e Governança

P2.9 RBR como pilar (narração)

- **Por quê:** RBR foi formalizado nos P0/P1, falta narrar no Cap. 2.
 - **Como:** seção “Pilares: RBR (FAST/DEEP) + Ω + PoP-L”.
 - **Entregável:** `/docs/cap2_arch.md` (adição de 1 subseção).
 - **Dono:** Eng.
 - **Aceitação:** descreve **critérios** e remete ao baseline Ω (P1.8).
-

Cap. 3 — Módulos Internos e API

P2.14 Sanitização/anonimização com metas

- **Por quê:** meta de PII 99,9% está no Cap. 6; ligar no Cap. 3.
- **Como:** inserir “Requisito de cobertura PII $\geq 99,9\%$ (por idioma/domínio)” com link ao KPI (P0.28).
- **Entregável:** `/docs/cap3_modules.md` (parágrafo curto).

- **Dono:** Eng + QA.
- **Aceitação:** meta aparece **textualmente** e com ID de KPI.

P2.20 Catálogo de exceções/falhas

- **Por quê:** consolidar erros e respostas padrão.
 - **Como:** tabela: code → módulo → mensagem → ação (retry, review, block).
 - **Entregável:** /docs/error_catalog.md.
 - **Dono:** Eng + SRE.
 - **Aceitação:** cobre ao menos: policy_not_found, invalid_rule, tsa_fail, plugin_timeout, schema_mismatch.
-

Cap. 5 — Conformidade, Segurança e Integridade

P2.26 SoD/dupla aprovação & human_override no DED

- **Por quê:** agora padronizado no P1.19; referenciar no Cap. 5.
- **Como:** nota normativa + snippet do campo no DED.
- **Entregável:** atualização em /docs/cap5_compliance.md.
- **Dono:** Compliance.
- **Aceitação:** DED mostra human_override e SoD exigido.

P2.27 Multi-tenancy declarado

- **Por quê:** RLS aparece no complemento; Cap. 5 deve **afirmar** isolamento.
 - **Como:** seção curta “Isolamento por tenant (RLS)”.
 - **Entregável:** mesma atualização do Cap. 5.
 - **Dono:** SRE/Eng.
 - **Aceitação:** RLS descrito + referência a tenantId nos schemas.
-

Cap. 6 — V&V e KPIs

P2.30 Testes de carga/soak e *circuit breaker* de plugins

- **Por quê:** plano de carga não estava formalizado.
- **Como:** definir perfis (pico/soak), thresholds e modo de degradação controlada.

- **Entregável:** `/tests/load_plan.md` + `/telemetry/plugins_circuit_breaker.md`.
- **Dono:** SRE + Eng.
- **Aceitação:** inclui cenários **FAST** e **DEEP**, metas compatíveis com SLOs (P1.25) e regras de *circuit breaker* integradas aos SLOs por plugin (P1.39).

P2.31 Métricas para factual/bias (FN/FP)

- **Por quê:** lacuna de métricas específicas.
- **Como:** adicionar **curvas por classe** (factual, bias) com alvos de FN/FP.
- **Entregável:** `/docs/kpi_bias_factual.md` (define targets e estratos).
- **Dono:** Data + QA.
- **Aceitação:** cada classe tem N mínimo, IC 95% e limites FN/FP explícitos.

P2.34 PDF/A com SLOs mínimos

- **Por quê:** relatório é assinado, falta metas operacionais embutidas.
- **Como:** anexar página “SLOs alcançados” com p95, lag Outbox, %CONFIRMED.
- **Entregável:** `template/report/pdfa_appendix_slos.md` (**INFERÊNCIA:** apêndice textual que vira PDF/A).
- **Dono:** QA + Compliance.
- **Aceitação:** PDF/A contém tabela SLOs e links de PoP-L.

Cap. 7 — Cripto e Algoritmos

P2.36 Guardrails do motor Ω (lint/consistência)

- **Por quê:** exigir *lint/teste* das regras (CEL/JSONLogic).
- **Como:** política: toda política passa por `lint` + `policyHash` antes de ativar.
- **Entregável:** `/policies/linting_rules.md`.
- **Dono:** Eng.
- **Aceitação:** PR que altera políticas **falha** sem `lint` (check CI).

P2.37 Backoff com limites/telemetria no Cap. 7

- **Por quê:** já definimos no P1.13; só referenciar no Cap. 7.
- **Como:** incluir seção “Backoff canônico: `max_retries/budget/telemetria`”.

- **Entregável:** atualização em `/docs/cap7_crypto_algos.md`.
- **Dono:** Eng.
- **Aceitação:** parâmetros batem com `retry_policy.md`.

P2.38 Suite anti-ReDoS (regex)

- **Por quê:** RE2 exigido, falta corpus/limites.
 - **Como:** documento com **corpus adversarial**, limites de tamanho/tempo e teste de *catastrophic backtracking*.
 - **Entregável:** `/security/regex_hardening.md`.
 - **Dono:** Eng + SRE.
 - **Aceitação:** CI roda suite e bloqueia regex não-re2 ou acima de `re_time_ms`.
-

Novas pendências (coesão de DB)

P2.44 TsaOutbox — onDelete: Cascade

- **Por quê:** evitar *dangling rows*.
 - **Como:** migração DB padronizando `ON DELETE CASCADE` para `AuditLog` → `TsaOutbox`.
 - **Entregável:** `migration_outbox_cascade.sql` (SQL simples).
 - **Dono:** Eng/DBA.
 - **Aceitação:** FK com CASCADE criada e verificada; testes de *replay* atualizados.
-

Sequenciamento sugerido (rápido e seguro)

1. Documentos narrativos do Cap. 1 → P2.1–2–3–4–5
2. Ajustes de arquitetura → P2.9
3. Camada de módulos/erros → P2.14 + P2.20
4. Conformidade e segurança → P2.26–27 + P2.38 + P2.44
5. V&V e relatórios → P2.30–31–34
6. Cripto/algoritmos → P2.36–37

Racional: minimizar retrabalho de referências; documentos do Cap. 1 e 2 “ancoram” o resto, e as migrações leves (P2.44) entram quando a doc já está alinhada.

Check de saída (para cada P2)

- **Documento criado/atualizado** no caminho indicado.
 - **Citações internas** para schemas/artefatos existentes (nada inventado).
 - **Hashes no PoP-L** (consistência com o dossiê).
 - **Sem divergências** com P0/P1 (FSM, enums, SLOs, baseline Ω , OpenAPI congelado).
-

perfeito — vou **executar a trilha 1 (Cap. 1)** agora e já te entrego os **cinco** artefatos P2.1–P2.5, prontos para colocar no repositório e no dossiê. Onde houver algo não literalmente nomeado nos PDFs, marco como **INFERÊNCIA** (sempre conservadora e coerente com o que já está nos seus arquivos).

/docs/cap1_context.md — Contexto & Fronteiras (Cap. 1) (P2.1 + P2.4 + P2.5)

1) Visão de alto nível (diagrama de contexto)

```
flowchart LR
    U[Usuário/App Cliente] -- prompt/resposta --> LLM[LLM]
    LLM -- chamada de mediação --> TER[TER-Kai Middleware]
    subgraph TER-Kai
        L0[L0 Kai (PII/normalização)]
        L1[L1 MIR (plugins/telemetria)]
        L2[L2 Ω (políticas/RBR)]
        L3[L3 Retro (revisão/recorreção)]
        L4[L4 Ledger (PoP-L/TSA)]
    end
    end
    TER -->|/api/v1/chat/invoke| L0
    TER -->|/api/v1/policies| L2
    TER -->|/api/v1/review-queue| L3
    TER -->|/api/v1/logs| L4
```

Interfaces expostas

- `POST /api/v1/chat/invoke` – entrada/saída de mensagens mediadas.
- `GET/POST /api/v1/policies` – gestão e versionamento de políticas (Ω).
- `GET /api/v1/logs[?tsa_status=]` e `GET /api/v1/logs/:id` – consulta DED/PoP-L.
- `GET/POST /api/v1/review-queue` – fila de revisão humana (HITL).
(todos já congelados no OpenAPI – ver P1.45).

2) Premissas operacionais testáveis (ligadas a V&V) (P2.4)

Premissa (Cap. 1)	KPI de validação	Caso V&V (ID)	Critério de aceitação	Evidência (PoP-L)
PII não vaza em	Cobertura PII ≥	GS-PII-003	IC 95% cobre a	hash TSA do

Premissa (Cap. 1)	KPI de validação	Caso V&V (ID)	Critério de aceitação	Evidência (PoP-L)
produção	99,9% por idioma		meta em todos estratos	relatório V&V
Roteamento seguro	FPR ≤ 1%	GS-RBR-004	FPR ≤ 1% por classe	DED de execução + baseline Ω
Baixa latência FAST	P95 < 800 ms	GS-PERF-FAST	p95 < 800 ms (1k req)	métrica assinada (PDF/A)
Correções automáticas predominam	≥ 80% dos casos	GS-RETRO-001	proporção ≥ 80% (IC 95%)	relatório V&V

Tudo já definido nos P0.28 (plano estatístico) e refletido no pipeline de gates (P1.32).

3) Papéis, RBAC e SoD (Separação de Deveres) (P2.5)

Papel (enum)	Permissões-chave	Restrições SoD
AUDITOR	ler / logs, gerar relatórios	não altera políticas
ENGINEER	publicar versão do app/policy draft	não aprova a própria mudança
COMPLIANCE	aprovar políticas, revisar HITL	não faz deploy
ADMIN	gerir RBAC/tenants	não atua em HITL cotidiano

- **HITL**: decisões via `/api/v1/review-queue` com `human_override=true` no DED (P1.19).
- **SoD**: quem cria/propõe **não aprova** (cap. 5 atualizado em P2.26).
- **Enum Role** já migrado (P1.42).

4) Referências cruzadas

- **Schemas canônicos**: `contracts-v1.0.0` (P1.7).
- **Baseline Ω**: `baseline-v1.0.0` (JSONLogic/CEL – P1.8).
- **OpenAPI congelado**: versão 1.0 (P1.45).

/docs/metrics_catalog.md — Catálogo de Métricas Externas (P2.2)

Lista as métricas/sinais consumidos pelo MIR (L1), suas origens, formatos e o **hash de referência** para reprodutibilidade (registrado no PoP-L).

ID	Métrica	Origem	Formato/Schema	Validação de entrada	Hash de referência	Atualização
M-PII	Detector PII	plugin	KaIPacket	regex segura,	sha256:	contínua

ID	Métrica	Origem	Formato/ Schema	Validação de entrada	Hash de referência	Atualização
		interno (RE2)	+ flags	limites de tamanho	< . . . >	
M-TOX	toxicity_index	modelo auxiliar	{score: [0,1]}	faixa [0,1], IC disponível? (INFERÊNCIA A)	sha256: < . . . >	diária
M-BIAS	bias_score	heurística/ ML	{score: [0,1]}	testes contra golden set bias	sha256: < . . . >	semanal
M-TRUST	trust_score	heurística	{score: [0,1]}	consistência por domínio	sha256: < . . . >	semanal
M-FACT	factual_risk	verificador factual	{score: [0,1]}	taxa FN/FP alvo (P2.31)	sha256: < . . . >	semanal

INFERÊNCIA: periodicidades sugeridas; ajuste conforme operação real.

Obrigatório: toda atualização de métrica gera **artefato versionado** com hash + TSA no PoP-L.

/docs/glossario.md — Glossário com Âncoras Canônicas (P2.3)

Cada termo aponta para um schema \$id do **catálogo de contratos** e, quando aplicável, para uma enum consolidada.

Termo	Definição	Âncora canônica
Kai (L0)	estágio de entrada/PII/normalização	\$id: KaiPacket
MIR (L1)	agregação de métricas/plugins	\$id: MetricsAgg
Ω Engine (L2)	avaliação de políticas/decisão	\$id: DecisionEvent, enum Decision
Retro (L3)	revisão/recorreção + revalidação	\$id: RevisedOutput
Ledger / PoP-L (L4)	registro imutável com TSA	\$id: LedgerDED, enum TsaStatus
RBR	roteamento por risco (FAST/DEEP)	enum Mode
Baseline Ω	política default (CEL/JSONLogic)	PolicyConfig + policyHash
HITL	revisão humana auditada	/api/v1/review-queue, human_override

Versão do conjunto de contratos: contracts-v1.0.0 (P1.7).

/docs/cap1_premissas_testaveis.md — Premissas & Testes V&V (âncora Cap. 1) (P2.4)

Documento enxuto para o Cap. 1 apontar **explicitamente** para os testes e KPIs correspondentes.

- **P-01** “O sistema bloqueia PII em produção”
 - KPI: $PII \geq 99,9\%$ (por idioma/domínio)
 - Teste: GS-PII-003
 - Evidência: relatório V&V com TSA (/reports/vv_*.json + PDF/A)
- **P-02** “O roteamento por risco minimiza falsos positivos”
 - KPI: $FPR \leq 1\%$ (estratos)
 - Teste: GS-RBR-004
 - Evidência: baseline Ω + execução assinada
- **P-03** “O caminho FAST é responsivo”
 - KPI: $p95 < 800\text{ ms}$
 - Teste: GS-PERF-FAST (1k req)
 - Evidência: métricas em PDF/A + PoP-L
- **P-04** “A maioria das correções é automática”
 - KPI: $\geq 80\%$
 - Teste: GS-RETRO-001
 - Evidência: V&V com seed, model_id, commit, environment_hash (P1.33)

(Todos os IDs e metas já definidos no P0.28/P1.33; aqui só consolidamos.)

/docs/cap1_roles_rbac.md — Papéis, RBAC e SoD (âncora Cap. 1) (P2.5)

Complemento do Cap. 1 focado em **quem pode o quê**, com separação de deveres e referências aos endpoints.

Matriz resumida

Recurso/ação	AUDITOR	ENGINEER	COMPLIANCE	ADMIN
Ler DED / logs	✓	✓	✓	✓
Criar/editar política	—	✓ (draft)	✓ (aprova)	✓

Recurso/ação	AUDITOR	ENGINEER	COMPLIANCE	ADMIN
Aprovar política	—	—	✓	✓
HITL decidir (/review-queue)	—	—	✓	✓
Deploy	—	✓ (com gates)	—	✓
Alterar log level prod	—	—	—	✓ (com SRE)

SoD mínimo:

- Quem edita política **não aprova**.
- Quem aprova **não faz deploy**.
- Decisão HITL sempre gera `human_override=true` no DED (P1.19).

Compatibilidade: enum Role consolidado (P1.42) e OpenAPI 1.0 (P1.45).

Critérios de aceitação (Trilha Cap. 1 — P2.1 a P2.5)

- Arquivos **criados**:
 - `/docs/cap1_context.md` (com diagrama + interfaces + RBAC/SoD resumidos)
 - `/docs/metrics_catalog.md`
 - `/docs/glossario.md`
 - `/docs/cap1_premissas_testaveis.md`
 - `/docs/cap1_roles_rbac.md`
- Todos **apontam** para schemas/artefatos já existentes (P0/P1).
- **Nenhuma divergência** com OpenAPI congelado, baseline Ω , FSM e enums.
- Metas/KPIs **iguais** às do plano estatístico (P0.28) e reprodutibilidade (P1.33).
- Onde há **INFERÊNCIA**, está marcado e é compatível com o que os arquivos estabelecem.

/docs/cap2_arch.md — P2.9 • RBR como Pilar Arquitetural

Objetivo

Formalizar o **roteamento por risco (RBR)** como componente de primeira classe da arquitetura TER-Kai, estabelecendo seus princípios, critérios, relação com o mecanismo Ω e impacto operacional.

Contexto e Motivação

O TER-Kai atua como middleware entre o LLM e o usuário, dividindo o fluxo em estágios L0 → L4.

O RBR define **como cada requisição percorre** esses estágios segundo o **nível de risco** avaliado no momento da entrada.

Princípio → Processamento proporcional ao risco: conteúdo benigno deve seguir o caminho FAST, enquanto conteúdo incerto ou sensível é roteado para DEEP ou REVIEW.

Camadas e Roteamento (visão macro)

Modo	Descrição	Etapas executadas	Atores	Saída principal
FAST	fluxo padrão de baixa latência	L0 + L1 + Ω (FAST)	automático	resposta direta + log
DEEP	fluxo estendido, validações adicionais	L0–L2 com plugins de alto custo e L3 Retro	automático	resposta após re-avaliação
REVIEW	intervenção humana	L0–L3 + HITL (reviewer)	humano + Ω	decisão ALLOW/BLOCK no DED
BLOCK	decisão automática Ω	L0–L2	automático	bloqueio + log PoP-L

(FAST/DEEP/REVIEW/BLOCK são os valores da enum *Mode*, já definidos nos schemas canônicos – P1.7.)

CrITÉrios NumÉricos e Classificação de Risco

A decisão de roteamento é tomada pelo motor Ω com base nos indicadores agregados pelo MIR (L1).

Métrica	Limiar ($\leq$) → FAST	Limiar ($>$) → DEEP	Limiar ($\uparrow$ extremo) → REVIEW/BLOCK
toxicity_index	0.40	0.40 – 0.75	> 0.75
bias_score	0.30	0.30 – 0.60	> 0.60
factual_risk	0.25	0.25 – 0.50	> 0.50
trust_score	≥ 0.70	0.40 – 0.70	< 0.40

INFERÊNCIA: valores ajustados a partir dos targets de PII/FPR (P0.28) e baseline Ω (P1.8); devem ser recalibrados conforme os testes V&V.

Esses limiares ficam registrados no artefato `/policies/baseline_omega.json` (v1.0.0) com hash PoP-L.

Relação com o motor Ω e PoP-L

1. O MIR gera KaiPacket com métricas.

2. O Ω consulta o baseline Ω + limiares RBR e decide mode.
3. O DED (L4) armazena mode e decision, gera policyHash e hash SHA3-512 com TSA.
4. O PoP-L mantém o histórico e permite verificação CLI (P0.21/P0.35).

Governança e Evidência

- **Política base:** baseline_omega.json (versãoada e assinada).
- **Auditoria:** logs do PoP-L contêm mode, metrics, policyHash, tsa_status.
- **Revisões:** mudanças de limiares exigem aprovação dupla (ENGINEER + COMPLIANCE).
- **CI/CD:** pipelines (P1.32) rodam V&V com thresholds RBR como gates.

Diagrama de Fluxo RBR

flowchart TD

```
A[Entrada: KaiPacket] --> B[MIR: métricas agregadas]
B --> C[" $\Omega$ : decisão com baseline  $\Omega$ "]
C -->|mode=FAST| F1["L1 -> L2 FAST -> resposta"]
C -->|mode=DEEP| F2["L1 -> L2 DEEP -> Retro"]
C -->|mode=REVIEW| H1["HITL Review Queue"]
C -->|mode=BLOCK| X1["Decisão BLOCK -> PoP-L"]
```

CrITÉrios de Aceitação

- Documento cap2_arch.md contém esta seção e remete para baseline Ω .
- Enums e limiares **coerentes** com os schemas e políticas (sem duplicação).
- Cada mudança de limiar gera novo hash PoP-L e incrementa baseline_version.
- FSM (P0.16) e RBR operam de forma integrada (FAST → DEEP → REVIEW → BLOCK).

P2.14 · Sanitização / Anonimização com metas

Objetivo

Estabelecer metas quantitativas e procedimentos auditáveis para mascaramento de PII, de modo alinhado ao KPI global (99,9 %) e ao plano de validação (V&V).

Abordagem técnica

- Criar documento → `/privacy/pii_sanitization_spec.md`
- Padronizar o pipeline L1 (MIR) com três camadas:
 1. **maskPII** – regex/NER determinístico;
 2. **redactSensitive** – heurísticas baseadas em contexto (emails, IBAN, telefone);
 3. **verifyPII** – pós-verificação com detector reverso (recovery-test).

Metas quantitativas

Indicador	Meta mínima	Método de cálculo	Frequência de auditoria
Cobertura PII	$\geq 99,9 \%$	$(\# \text{ PII detectadas} - \# \text{ PII não mascaradas}) / \# \text{ PII detectadas}$	diária
FPR (falsos positivos)	$\leq 1 \%$	$\# \text{ tokens inocentes mascarados} / \# \text{ tokens avaliados}$	semanal
FNR (falsos negativos)	$\leq 0,1 \%$	$\# \text{ PII não detectadas} / \# \text{ PII rotuladas}$	diária

Essas metas são **referenciadas no Cap. 6 (KPIs)** e **testadas automaticamente** nos pipelines V&V (P0.28).

validação e evidência

- **golden set_privacy_v1.json**: amostra estratificada de PII.
- **PoP-L hash**: SHA3-512 + TSA.
- **CI/CD gate**: reprova o build se `coverage < 99.9 %`.

Critérios de aceite

- Documento `/privacy/pii_sanitization_spec.md` completo e versionado.
- Métricas replicadas no dashboard `/monitoring/slo_targets.md`.
- V&V executa relatório `pii_report.json` → PDF/A anexado no PoP-L.

P2.20 · Catálogo de Exceções / Falhas

Objetivo

Padronizar códigos de erro e respostas automáticas para falhas previsíveis, garantindo rastreabilidade e auditoria forense coerente com o Ledger.

Documento gerado: /docs/error_catalog.md

Código	Módulo	Causa	Severidade	Ação automática	Registro
policy_not_found	Ω / PolicyLoader	baseline ausente ou hash inválido	⚠ Alta	fallback → BLOCK	PoP-L entry + alert
tsa_fail	L4 / Ledger	falha de timestamp RFC 3161	⚠ Alta	retry × 3 → queue	Outbox + error_log
retro_timeout	L3 / Retro	tempo > budget ms	⚙ Média	retry/backoff	job trace
review_queue_empty	HITL	sem reviewers ativos	Média	fallback → DEEP	review_log
metrics_malformed	L1 / MIR	input sem schema válido	Média	discard + warn	metrics_audit
auditlog_corrupt	L4 / Ledger	hash não confere com PoP-L	Crítica	bloqueio + investigação	incident_report

Integração operacional

- Todos os erros geram evento no **AuditLog** (error_code, severity, timestamp).
- **PoP-L** encadeia o erro ao auditLogId.
- **Retry/alerting** tratados via Redis + Grafana.
- CI/CD (P1.32) injeta testes de erro sintéticos a cada release.

Critérios de aceite

- error_catalog.md publicado + importável via API (/api/v1/errors).
 - Scripts de teste simulando todas as falhas catalogadas.
 - Dashboard Grafana com alertas por severidade.
-

/docs/cap5_compliance.md — Capítulo 5 ·

Conformidade, Segurança e Integridade

Objetivo

Consolidar a política de **Separação de Deveres (SoD)** e o modelo de **isolamento multi-tenant** no núcleo do TER-Kai, para garantir rastreabilidade, auditabilidade e independência operacional.

P2.26 — SoD / Dupla Aprovação e human_override

Princípio geral

Toda decisão ou política que impacta comportamento do sistema deve exigir **duas aprovações de papéis distintos**, com rastreabilidade no **PoP-L**.

Nenhuma operação crítica deve depender de um único ator.

Fluxo HITL (Human-In-The-Loop)

Derivado do /docs/hitl_flow.md (P1.19).

flowchart LR

```
S1[Solicitação de revisão] --> S2[Reviewer Compliance]
S2 --> S3[Decisão: APPROVE / REJECT / ESCALATE]
S3 -->|APPROVE| S4[PoP-L: hash do DED + reviewer_id + TSA]
S3 -->|ESCALATE| S5[2º Reviewer Admin → decisão final]
```

Campos obrigatórios no DED (já aplicados):

```
{
  "human_override": true,
  "reviewer_id": "uuid",
  "decision": "APPROVE",
  "reason": "Justificativa do revisor",
  "timestamp": "2025-11-12T12:00:00Z"
}
```

Separação de deveres:

Etapas	Papel permitido	Restrição SoD
Submissão	ENGINEER / SYSTEM	não pode revisar
Revisão	COMPLIANCE / ADMIN	não pode submeter
Aprovação final	ADMIN	requer revisão anterior

Auditoria:

Cada revisão gera hash TSA (PoP-L) e é armazenada no Ledger.

Logs de HITL são protegidos contra alteração (`immutability: true`).

Política de aprovação dupla

- **Alterações de políticas** (`/api/v1/policies`) → requerem `reviewed_by` + `approved_by`.
- **Deploys de produção** → requerem `approved_by` ≠ `deployed_by`.
- **Exceções manuais** (flags de bypass) → exigem `human_override=true` e justificção.

Exemplo de evidência PoP-L:

```
{  
  "policy_id": "Ω-101",  
  "policy_hash": "sha3_512:abcd...",  
  "reviewed_by": "user_002",  
  "approved_by": "user_006",  
  "tsa_status": "CONFIRMED"  
}
```

Critérios de aceitação

- Campos `human_override`, `reviewer_id`, `decision` presentes e validados no schema.
 - Dupla aprovação exigida nas rotas `/api/v1/policies` e `/api/v1/deploy`.
 - PoP-L registra os dois revisores distintos.
 - Auditoria CLI (`verify-sod`) reconstrói cadeia de aprovação.
-

P2.27 — Multi-Tenancy Declarado (RLS)

Objetivo

Formalizar o isolamento de dados entre tenants, garantindo segregação total de logs, políticas e chaves criptográficas.

Cada tenant opera dentro de um “domínio de segurança isolado”, sem acesso a dados ou metadados de outros.

Implementação técnica

a) Esquema de banco de dados

Cada tabela crítica inclui o campo `tenant_id` e política de **Row-Level Security (RLS)**:

```
ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY;  
CREATE POLICY tenant_isolation_policy ON audit_logs  
USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

b) Contexto de execução

- O `tenant_id` é **injetado pelo middleware Kai** com base no token JWT ou header `X-Tenant-ID`.
- O identificador é validado antes de cada transação:
 - se ausente → erro `missing_tenant_context`;
 - se inválido → erro `unauthorized_tenant`.

c) Chaves e KMS

- Cada tenant possui par DEK único (ver P1.23).
- O KEK global apenas envelopa DEKs, sem acesso cruzado.

d) Telemetria e PoP-L

- Logs, métricas e políticas possuem `tenant_id`.
- As evidências TSA (PoP-L) são segregadas por namespace de tenant (PoP-L/<tenant_id>).

Exemplo de DED (auditável)

```
{  
  "ded_id": "uuid",  
  "tenant_id": "a3f5c1...",  
  "mode": "FAST",  
  "decision": "ALLOW",  
  "policy_hash": "sha3_512:...",  
  "tsa_status": "CONFIRMED"  
}
```

Critérios de aceitação

- Toda tabela de dados sensíveis possui `tenant_id` e política RLS habilitada.

- PoP-L e KMS segregam evidências/chaves por tenant.
- Middleware bloqueia requisições sem contexto de tenant.
- Teste de penetração: tenant A não acessa dados do tenant B.

Resumo final — Grupo 4 (Cap. 5)

Item	Tema	Entregável	Resultado esperado
26	SoD / HITL / dupla aprovação	/docs/cap5_compliance.md	auditoria imutável, 2 revisores distintos, human_override padronizado
27	Multi-tenancy (RLS)	idem	isolamento de tenants e segregação de logs/chaves

/tests/load_plan.md — Plano de Carga / Soak & Circuit Breaker (P2.30)

Objetivo

Validar desempenho/estabilidade do TER-Kai sob picos e longas durações, com degradação controlada (**circuit breaker**) alinhada aos SLOs (P1.25) e SLOs por plugin (P1.39).

1) Perfis de teste

Perfil	Duração	Alvo	RPS	Conexões	Observações
Smoke	5 min	FAST	50	50	sanity/telemetria ok
Baseline	30 min	FAST/DEEP 80/20	250	200	valida p95 FAST/DEEP
Pico	10 min	FAST/DEEP 60/40	1,000	600	saturação controlada
Soak	4 h	FAST/DEEP 70/30	200	200	memory leaks/retentativas
Chaos TSA	30 min	L4	150	150	5% falha TSA simulada
Plugins Burst	20 min	MIR	300	250	latências por plugin

Ferramenta sugerida: k6 ou vegeta (**INFERÊNCIA**). Scripts ficam em /tests/k6/*.js.

2) Alvos e SLOs de aprovação

Métrica	Target	Condição de aprovação
FAST p95	< 800 ms	em Baseline/Pico/Soak
DEEP p95	< 4 s	em Baseline/Soak
Outbox lag p95	< 60 s	em todos os perfis
TSA carimbo p95	< 3 min	em Chaos TSA
Error rate 5xx	< 1 %	qualquer perfil
Retry success	≥ 95 %	Chaos TSA/Soak

3) Circuit Breaker (plugins MIR)

- **Entrada:** latência p95/p99 e `timeout_ms` por plugin (`/telemetry/plugins_slos.md`).
- **Regra:** se `p95 > target` por **10 min** ou `timeout_total > 20/min`, **acionar breaker**.
- **Comportamento:**
 - **HIGH risk** → **fail-closed**: roteia para **REVIEW/BLOCK** conforme baseline Ω .
 - **LOW/MED** → **fail-open**: aplica *fallback* (`last_good/default_score`) e registra `plugin_fallback_total++`.

4) Sinóptico de gatilhos/ações

Evento	Gatilho	Ação	Evidência
BreakerOpen[plugin]	p95>target 10 min	ativar modo fail-*	DED flag plugin_failure
TsaDegraded	TSA p95>180s	aumentar backoff (P1.13)	métrica tsa_latency
OutboxBacklog	lag p95>60s	aumentar workers + throttling	métrica outbox_lag

5) Consultas Prometheus (exemplos)

```
histogram_quantile(0.95, sum(rate(kai_fast_latency_ms_bucket[5m])) by (le))
histogram_quantile(0.95, sum(rate(kai_deep_latency_ms_bucket[5m])) by (le))
max(outbox_lag_seconds_p95)
max(tsa_latency_seconds_p95)
sum(rate(mir_plugin_timeout_total[1m])) by (plugin)
```

6) Critérios de aceite

- Relatório `load_report.json` + PDF/A com tabelas p95, erro e SLOs.
- Logs de “breaker” presentes quando gatilhos ocorrerem.
- Nenhum leak de memória/FD em Soak (monitor via `container_memory_usage_bytes`).

/docs/kpi_bias_factual.md — KPIs de Factual/Bias (FN/FP, estratos e N/IC) (P2.31)

Objetivo

Definir métricas e alvos para **factualidade** e **viés**, com estratificação, tamanhos de amostra (N) e intervalos de confiança (IC 95%), integrados ao V&V.

1) Métricas principais

- **FN (False Negative):** risco **não** detectado como risco.
 - **FP (False Positive):** conteúdo seguro marcado como risco.
 - **Precision/Recall/F1** por classe (factual, bias).
 - **AUC/PR** (opcional).
-

2) Alvos por classe

Classe	FN máx	FP máx	F1 min
Factual	≤ 8 %	≤ 5 %	≥ 0.85
Bias	≤ 10 %	≤ 6 %	≥ 0.80

INFERÊNCIA: valores iniciais; calibrar via golden set e revisão de risco.

3) Estratos mínimos (amostragem)

- **Idioma:** pt / en / es (min. N=300/idioma).
- **Domínio de conteúdo:** tech / saúde / jurídico (min. N=200/domínio).
- **Comprimento do texto:** curto/medio/longo (min. N=150/estrato).
- **Sensibilidade PII:** baixa/média/alta (min. N=150/estrato).

N total recomendado: ≥ 2.400 exemplos.

4) Cálculo e IC (95%)

- IC por Wilson para proporções (FN/FP).
 - Reportar **média ponderada** e também **por estrato**.
 - A amostra deve ser **fixada por seed** (ver P1.33: seed, model_id, commit, environment_hash).
-

5) Formato de relatório (JSON)

```
{
  "report_id": "bias_fact_v1",
  "generated_at": "2025-11-12T12:00:00Z",
  "model_id": "provider/model",
  "seed": 42,
  "commit": "abcdef1",
  "environment_hash": "sha256:...",
  "golden_set_version": "gs-bias-fact-v1",
  "metrics": {
    "factual": { "FN": 0.075, "FP": 0.042, "F1": 0.86, "IC95_FN":
[0.060,0.090] },
    "bias": { "FN": 0.092, "FP": 0.055, "F1": 0.82, "IC95_FN":
[0.078,0.108] }
  },
  "by_strata": [ { "lang":"pt", "FN":0.07, "FP":0.04 }, ... ],
  "tsa_token": "RFC3161_token",
  "payload_hash_sha3_512": "sha3_512:..."
}
```

6) Gates de aprovação (CI/CD)

- Reprovar PR/deploy se **qualquer** estrato violar os limites de FN/FP.
- Emitir `policy_tuning_required` se F1 cair > **3 p.p.** em relação à baseline do PoP-L.

7) Critérios de aceite

- Documento publicado e referenciado no Cap. 6.
 - Relatório V&V inclui seções **globais e por estrato** com IC 95%.
 - Hash + TSA do relatório no PoP-L.
-

/report/pdfa_appendix_slos.md — Apêndice de SLOs no PDF/A (P2.34)

Objetivo

Incluir, nos relatórios PDF/A assinados, uma página padrão que mostre os SLOs alcançados na execução (ligados ao PoP-L).

Estrutura do apêndice

Título: “Apêndice A — SLOs Operacionais”

Tabela 1 — SLOs alcançados

Métrica	Target	Observado (p95)	Janela	Status
Outbox lag	< 60 s	41 s	30 min	✓
TSA carimbo	< 3 min	122 s	30 min	✓
FAST latência	< 800 ms	735 ms	30 min	✓
DEEP latência	< 4 s	3.2 s	30 min	✓
Erro 5xx	< 1 %	0.8 %	30 min	✓

Notas:

- fonte: métricas Prometheus do intervalo da execução (hash/ID do snapshot).
- cada linha aponta para um **artefato de evidência** (CSV/JSON) com **hash SHA3-512**.

Bloco de evidência PoP-L:

```
payload_hash_sha3_512: sha3_512:...  
tsa_token: (base64 RFC3161)  
snapshot_id: ops-2025-11-12T12:00:00Z
```

Assinatura:

- reviewer: COMPLIANCE (SoD com ENGINEER).
- verificação via CLI `verify-pdf a --appendix-slos`.

Critérios de aceite

- Todo relatório PDF/A contém o apêndice com **tabela preenchida e hashes**.
 - O apêndice é **consistente** com SLOs definidos em `/monitoring/slo_targets.md`.
 - Validação automatizada em CI garante presença e consistência dos números.
-

Resumo — Grupo 5 (Cap. 6) concluído

Item	Entregável	Resultado
P2.30	/tests/load_plan.md	Perfis de carga/soak + circuit breaker operacional
P2.31	/docs/kpi_bias_factual.md	KPIs por classe com N/IC e gates
P2.34	/report/ pdfa_appendix_slos.md	Apêndice padrão com SLOs e PoP-L

/policies/linting_rules.md — P2.36 · Guardrails do motor Ω (lint/consistência)

Objetivo

Impedir que políticas (CEL/JSONLogic) inconsistentes, não determinísticas ou inseguras sejam ativadas. O *lint* roda no CI (pré-merge) e no *pre-activate* da rota `/api/v1/policies`.

1) Regras de lint (universais)

- **Determinismo:** funções proibidas de aleatoriedade/tempo (**INFERÊNCIA:** p.ex. `now()`, `rand()`).
- **Inputs fechados:** apenas variáveis do `KaiPacket` e métricas do MIR (catálogo P2.2).
- **Sem efeitos colaterais:** política não pode emitir I/O.
- **Tipos estritos:** comparação numérica só com números; `true/false` explícitos.
- **Cobertura de decisão:** toda condição deve terminar em `{ALLOW|REVIEW|BLOCK}`.
- **Limites:** profundidade de árvore ≤ 20 ; nós totais ≤ 1.000 (**INFERÊNCIA** de limite seguro).
- **Hash:** `policyHash = sha256(stableStringify(rules))` (P1.40).

2) Regras específicas — JSONLogic

- Operadores permitidos: `if`, `and`, `or`, `!`, `>=`, `>`, `<=`, `<`, `==`, `in`, `all`, `none`, `some`.
- Proibidos: `cat`, `map`, `reduce` (se alterarem determinismo) — **INFERÊNCIA** conservadora.
- Validação estrutural via JSON Schema (`PolicyConfig` do catálogo v1.0.0).

3) Regras específicas — CEL

- Biblioteca reduzida: somente tipos primitivos e `math` básico; sem libs externas.
- Proibido acesso a tempo/ambiente/processo.

- *Static type check* obrigatório (sem casts dinâmicos ambíguos).
-

4) Exemplos de *lint failures*

- Uso de variável não declarada: `bias` em vez de `bias_score`.
 - Divisão por zero potencial (operador `/?` sem guarda) — **INFERÊNCIA**.
 - Ramo sem retorno: cadeia `if` que não termina em decisão.
-

5) Saída do linter (formato)

```
{
  "policy_id": "baseline-v1.0.1",
  "engine": "CEL",
  "status": "FAILED",
  "errors": [
    { "code": "UNDECLARED_VAR", "path": "$.expr[2]", "msg": "bias não declarado;
use bias_score" },
    { "code": "NON_DETERMINISM", "path": "$.expr[5]", "msg": "uso de função
now() proibida" }
  ],
  "policyHash": "sha256:..."
}
```

6) Gates de ativação

- PR falha se `status != PASSED`.
- `/api/v1/policies/activate` retorna 409 com relatório do linter.

Critérios de aceitação

- Linter rodando no CI e no *pre-activate*.
 - Bloqueio de políticas não determinísticas ou fora do schema.
 - `policyHash` gerado com **`stableStringify`** (P1.40).
-

/docs/cap7_crypto_algos.md (addenda) — P2.37 ·

Backoff com limites/telemetria

Esta seção referencia e **ancora no Cap. 7** os parâmetros já definidos em /docs/retry_policy.md (P1.13), para que o capítulo criptográfico/algorítmico contenha o comportamento canônico.

1) Parâmetros canônicos (referência)

Parâmetro	Valor	Observação
base_delay_ms	200	Backoff exponencial
exponent	2	Crescimento quadrático
jitter	±30%	Evita <i>thundering herd</i>
max_retries	5	Limite duro
max_budget_ms	5000	Orçamento total

Telemetria obrigatória: `retry_count`, `retry_success_rate`, `tlsa_latency_seconds_p95`.

Estados alinhados: QUEUED/PROCESSING/CONFIRMED/FAILED (P0.41/P1.45).

2) Comportamento formal

- **Erros recuperáveis** → aplicam backoff + jitter até `max_retries` ou `max_budget_ms`.
- **Irrecuperáveis** → falha imediata (VALIDATION_ERROR, SCHEMA_MISMATCH).
- **Registro:** cada tentativa incrementa `retry_count` no DED e exporta métrica.

3) Prova e auditoria

- Cada transição de estado gera DED assinado (PoP-L), acoplado ao `auditLogId`.
- As métricas de `retry_success_rate` entram no apêndice PDF/A (P2.34).

Critérios de aceitação

- Cap. 7 contém esta referência **coerente** com `retry_policy.md`.
- Métricas e estados visíveis no Prometheus/Grafana conforme P1.25.

/security/regex_hardening.md — P2.38 · Suite anti-ReDoS (regex)

Objetivo

Estabelecer limites e testes adversariais contra *catastrophic backtracking* em qualquer regex usada pelos módulos (principalmente PII e parsers).

1) Padrões técnicos

- **Engine:** RE2 ou equivalente *time-safe* (sem backtracking exponencial).
- **Limites:**
 - comprimento de entrada: $\leq 64 \text{ KiB}$ por campo analisado — **INFERÊNCIA** conservadora;
 - tempo máximo por avaliação: $\leq 5 \text{ ms}$ p95 e $\leq 20 \text{ ms}$ p99;
 - número de grupos de captura: ≤ 20 .
- **Proibição:** *lookbehinds* e quantificadores aninhados patológicos (em engines que permitiriam).

2) Corpus adversarial (exemplos)

- Strings com repetições longas: "aaaaaaaaa...b" ($10^5 a + b$).
- Padrões densos: $\wedge(a+)+\$, ((ab)^*)^*$ — servem apenas para teste/negação.
- Campos típicos: e-mail, telefone, CPF/CNPJ, IBAN — com ruído inserido.

3) Testes automatizados (CI)

Scripts em `/tests/regex/`:

1) valida engine e flags permitidas

```
npm run regex:lint
```

2) roda suite adversarial, mede p95/p99

```
npm run regex:fuzz -- --report regex_report.json
```

3) gate de aprovação

```
node scripts/regex_gate.js regex_report.json --p95-ms 5 --p99-ms 20
```

4) Relatório (formato)

```
{
  "engine": "RE2",
  "patterns_tested": 42,
  "p95_eval_ms": 3.7,
  "p99_eval_ms": 12.1,
  "max_input_len": 65536,
  "violations": [
    { "pattern": "^(a+)+$", "reason": "backtracking", "status": "blocked" }
```

```
],  
"tsa_token": "RFC3161_token",  
"payload_hash_sha3_512": "sha3_512:..."  
}
```

5) Integração com catálogo de erros

- Violação gera `regex_unsafe_pattern` em `/docs/error_catalog.md` (P2.20).
- PR é bloqueado; política de *override* exige **dupla aprovação** (P2.26) e marca `human_override=true`.

Critérios de aceitação

- Suite ativa no CI, com *gate* de p95/p99 e tamanho de entrada.
- Toda regex do projeto passa por lint + fuzz adversarial.
- Relatórios assinados (PoP-L) e vinculados ao PDF/A quando aplicável.

Status Cap. 7 — Concluído (P2.36 · P2.37 · P2.38)

- Lint obrigatório para políticas Ω (CEL/JSONLogic).
- Parâmetros de backoff ancorados no capítulo (consistentes com P1.13).
- Suite anti-ReDoS operacional e integrada ao catálogo de erros/SoD.

migration_outbox_cascade.sql — Migração PostgreSQL (segura e idempotente)

```
-- MIGRATION: Padroniza FK TsaOutbox -> AuditLog com ON DELETE CASCADE  
  
BEGIN;  
  
-- 0) Sanidade: não pode existir outbox sem pai (corrige antes de criar CASCADE)  
-- (Se retornar linhas, trate-as conforme política: apagar, reanexar ao pai  
-- correto, ou arquivar.)  
WITH orphans AS (  
    SELECT o.id AS outbox_id  
    FROM "TsaOutbox" o  
    LEFT JOIN "AuditLog" a ON a.id = o."auditLogId"
```

```

WHERE a.id IS NULL
)
SELECT COUNT(*) AS orphan_count FROM orphans;

-- 1) Descobrir e dropar a FK atual (qualquer nome)
DO $$
DECLARE
    fk_name text;
BEGIN
    SELECT conname
    INTO fk_name
    FROM pg_constraint c
    JOIN pg_class t ON t.oid = c.conrelid
    WHERE t.relname = 'TsaOutbox'
        AND c.contype = 'f'
        AND pg_get_constraintdef(c.oid) ILIKE '%FOREIGN KEY ("auditLogId")
REFERENCES "AuditLog"%';

    IF fk_name IS NOT NULL THEN
        EXECUTE format('ALTER TABLE "TsaOutbox" DROP CONSTRAINT %I;', fk_name);
    END IF;
END$$;

-- 2) (Opcional) Índice de apoio à FK (se não existir)
CREATE INDEX IF NOT EXISTS "idx_tsaoutbox_auditlogid" ON "TsaOutbox"
("auditLogId");

-- 3) Recriar FK com ON DELETE CASCADE
ALTER TABLE "TsaOutbox"
ADD CONSTRAINT "fk_tsaoutbox_auditlog_cascade"
FOREIGN KEY ("auditLogId")
REFERENCES "AuditLog" ("id")
ON DELETE CASCADE;

COMMIT;

```

Observação: o passo 0 lista órfãos; a migração **não** os apaga automaticamente para você decidir o destino (compliance). Se quiser automatizar a limpeza: `DELETE FROM "TsaOutbox" USING ... WHERE a.id IS NULL;`

Rollback seguro (se precisar reverter)

```
BEGIN;

ALTER TABLE "TsaOutbox" DROP CONSTRAINT IF EXISTS
"fk_tsaoutbox_auditlog_cascade";

ALTER TABLE "TsaOutbox"

  ADD CONSTRAINT "fk_tsaoutbox_auditlog_noaction"

  FOREIGN KEY ("auditLogId")

  REFERENCES "AuditLog" ("id")

  ON DELETE NO ACTION;

COMMIT;
```

Ajuste no ORM (Prisma/...)

Garanta consistência no nível do ORM para evitar *drift* futuro:

```
model AuditLog {
  id          String    @id @default(uuid())
  outboxes    TsaOutbox[]
}

model TsaOutbox {
  id          String    @id @default(uuid())
  auditLogId  String
  auditLog    AuditLog @relation(fields: [auditLogId], references: [id],
onDelete: Cascade)
}
```

Runbook de validação e *replay*

Pré-deploy (staging)

1. **Backup:** snapshot do DB.
2. **Checar órfãos** (passo 0 da migração). Decidir destino (apagar/relocar).

3. **Corte de tráfego** (curto): pausar o **TSA Worker** para evitar corridas.

Deploy

4. Aplicar `migration_outbox_cascade.sql`.

5. Conferir a FK:

```
SELECT conname, pg_get_constraintdef(c.oid)
FROM pg_constraint c
JOIN pg_class t ON t.oid = c.conrelid
WHERE t.relname = 'TsaOutbox' AND c.contype = 'f';
```

Deve mostrar ON DELETE CASCADE.

Testes funcionais (staging)

6. Teste 1 — cascata básica

- Criar AuditLog A1 e 2 filhos em TsaOutbox.
- `DELETE FROM "AuditLog" WHERE id = 'A1';`
- Esperado: 0 linhas em TsaOutbox para auditLogId='A1'.

7. Teste 2 — replay de TSA

- Criar AuditLog A2 + 1 filho outbox (QUEUED).
- Rodar **TSA Worker** → CONFIRMED.
- `DELETE FROM "AuditLog" WHERE id = 'A2';`
- Esperado: todas as linhas de outbox e quaisquer retries relacionados **somem**; worker **não** reprocessa A2.

8. Teste 3 — purga em massa (LGPD/retention)

- Apagar 100 AuditLog antigos.
- `SELECT COUNT(*) FROM "TsaOutbox" WHERE "auditLogId" IN (...);` → 0.

Pós-deploy (produção)

9. Reativar TSA Worker.

10. Monitorar por 24h:

- `mir_plugin_errors_total` (drifts inesperados)
- `outbox_lag_seconds_p95` (não deve subir)
- `tsa_latency_seconds_p95` (estável)

Impacto e compatibilidade

- **Leitura/consulta:** sem impacto.
- **Escrita:** deletar `AuditLog` agora remove outboxes dependentes automaticamente → elimina *leaks* de fila.
- **Workers:** o TSA Worker deve validar a existência do pai ao consumir mensagens; se usar *polling* direto na tabela, adicionar `JOIN AuditLog` para evitar pegar algo já cascated.

Critérios de aceitação (P2.44)

- `FK TsaOutbox(auditLogId) → AuditLog(id)` com **ON DELETE CASCADE** confirmada no catálogo do DB.
 - Zero órfãos após a migração.
 - Testes 1–3 **aprovados** em staging.
 - Métricas de lag/latência TSA estáveis após 24h.
 - ORM atualizado com `onDelete: Cascade` na relação.
-

Release Pack — RLS-PoP-Enums-Outbox

Objetivo do pacote

- **Congelar estado** de schema/contratos após P1+P2.
- **Executar migrações em ordem segura** (idempotentes).
- **Registrar evidências** (hashes, TSA) no PoP-L.
- **Harmonizar Dev/Stage/Prod** com o mesmo playbook.

Estrutura sugerida de diretórios

release/

 v2_0/

 MIGRATE_ORDER.txt

 preflight.sql

 01_role_enum.sql

 02_policy_domain_unique.sql

 03_rls_enable_tenant.sql

 04_outbox_ondetelete_cascade.sql

 verify.sql

 rollback/

 01_role_enum_rollback.sql

 02_policy_domain_unique_rollback.sql

 03_rls_disable_tenant.sql

 04_outbox_ondetelete_noaction.sql

 docs/

 RELEASE_NOTES.md

 CHANGELOG.md

 EVIDENCE_PoP-L.json

 ci/

 github-actions.yml

 scripts/

 run_release.sh

 compute_hashes.sh

Ordem de execução (MIGRATE_ORDER.txt)

1. preflight.sql
2. 01_role_enum.sql
3. 02_policy_domain_unique.sql
4. 03_rls_enable_tenant.sql
5. 04_outbox_ondelete_cascade.sql
6. verify.sql

Pré-checks — preflight.sql (idempotente)

```
-- Fail-fast se não estiver em transação controlada
SHOW transaction_isolation;

-- 1) Health básico
SELECT version();

-- 2) Verificar presença das tabelas críticas
SELECT to_regclass('public."AuditLog"') IS NOT NULL AS has_auditlog,
       to_regclass('public."TsaOutbox"') IS NOT NULL AS has_outbox;

-- 3) Drift de enums (somente inspeciona; não altera)
--   Ajustar nomes se os seus objetos tiverem schema diferente.
SELECT t.typname AS enum, e.enumlabel AS value
FROM pg_type t
JOIN pg_enum e ON t.oid = e.enumtypid
WHERE t.typname IN ('Role', 'Decision', 'Mode', 'TsaStatus')
ORDER BY t.typname, e.enumsortorder;

-- 4) Checar colunas de tenant e índices mínimos (avisa; corrige na migração 03)
SELECT column_name
FROM information_schema.columns
WHERE table_name='AuditLog' AND column_name='tenant_id';
```

```
-- 5) Procurar órfãos de outbox (migr. 04 trata manualmente se existirem)
WITH orphans AS (
    SELECT o.id
    FROM "TsaOutbox" o
    LEFT JOIN "AuditLog" a ON a.id = o."auditLogId"
    WHERE a.id IS NULL
)
SELECT COUNT(*) AS orphan_outbox FROM orphans;
```

1) Role como Enum — 01_role_enum.sql

Padroniza `User.role` para **ENUM Role** com valores canônicos; migra valores texto.

```
BEGIN;

-- Criar enum se não existir
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'Role') THEN
        CREATE TYPE "Role" AS ENUM ('AUDITOR', 'ENGINEER', 'COMPLIANCE', 'ADMIN');
    END IF;
END$$;

-- Adicionar coluna temporária do tipo enum
ALTER TABLE "User" ADD COLUMN IF NOT EXISTS role_enum "Role";

-- Migrar valores existentes (“melhor esforço”)
UPDATE "User" SET role_enum =
    CASE UPPER(role)
        WHEN 'AUDITOR' THEN 'AUDITOR'::"Role"
        WHEN 'ENGINEER' THEN 'ENGINEER'::"Role"
        WHEN 'COMPLIANCE' THEN 'COMPLIANCE'::"Role"
        WHEN 'ADMIN' THEN 'ADMIN'::"Role"
        ELSE 'AUDITOR'::"Role" -- default conservador
    END
WHERE role_enum IS NULL;
```

```
-- Trocar colunas
ALTER TABLE "User" DROP COLUMN IF EXISTS role;
ALTER TABLE "User" RENAME COLUMN role_enum TO role;

COMMIT;

Rollback — rollback/01_role_enum_rollback.sql
BEGIN;
ALTER TABLE "User" ADD COLUMN IF NOT EXISTS role_text TEXT;
UPDATE "User" SET role_text = role::text;
ALTER TABLE "User" DROP COLUMN role;
ALTER TABLE "User" RENAME COLUMN role_text TO role;
DROP TYPE IF EXISTS "Role";
COMMIT;
```

2) Unicidade de Policy por domínio/versão — 02_policy_domain_unique.sql

Evita duplicidade de políticas ativas por domínio; recomendo chave composta.

```
BEGIN;

-- Se existir índice antigo @unique(domain), remove para criar a composta
DO $$
DECLARE idx text;
BEGIN
    SELECT indexname INTO idx
    FROM pg_indexes
    WHERE tablename='PolicyConfig' AND indexdef ILIKE '%(domain)%' AND indexdef
    ILIKE '%UNIQUE%';
    IF idx IS NOT NULL THEN
        EXECUTE format('DROP INDEX IF EXISTS %I;', idx);
    END IF;
END$$;

-- Criar chave composta (domain, version)
```

```
DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM pg_indexes
        WHERE tablename='PolicyConfig'
            AND indexdef ILIKE '%UNIQUE%'
            AND indexdef ILIKE '%(domain, version)%'
    ) THEN
        ALTER TABLE "PolicyConfig" ADD CONSTRAINT policy_domain_version_unique
        UNIQUE ("domain","version");
    END IF;
END$$;
```

```
COMMIT;
```

Rollback — rollback/02_policy_domain_unique_rollback.sql

```
BEGIN;
ALTER TABLE "PolicyConfig" DROP CONSTRAINT IF EXISTS
policy_domain_version_unique;
-- (Opcional) recriar @unique(domain) se era o padrão anterior
-- CREATE UNIQUE INDEX IF NOT EXISTS policy_domain_unique ON
"PolicyConfig"(domain);
COMMIT;
```

3) RLS / Multi-tenant habilitado — 03_rls_enable_tenant.sql

Liga Row-Level Security nas tabelas sensíveis e cria política baseada em `app.tenant_id`.

```
BEGIN;

-- Habilitar RLS (ajuste lista de tabelas conforme seu schema)
ALTER TABLE "AuditLog"          ENABLE ROW LEVEL SECURITY;
ALTER TABLE "TsaOutbox"         ENABLE ROW LEVEL SECURITY;
ALTER TABLE "AuditLogDebug"     ENABLE ROW LEVEL SECURITY;
ALTER TABLE "PolicyConfig"      ENABLE ROW LEVEL SECURITY;
```

```

-- Política genérica por tenant
DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM pg_policies WHERE schemaname='public' AND tablename='AuditLog'
        AND policyname='tenant_isolation_policy'
    ) THEN
        CREATE POLICY tenant_isolation_policy ON "AuditLog"
            USING ("tenant_id" = current_setting('app.tenant_id')::uuid);
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM pg_policies WHERE schemaname='public' AND
        tablename='TsaOutbox' AND policyname='tenant_isolation_policy'
    ) THEN
        CREATE POLICY tenant_isolation_policy ON "TsaOutbox"
            USING ("auditLogId" IN (SELECT id FROM "AuditLog" WHERE "tenant_id" =
            current_setting('app.tenant_id')::uuid));
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM pg_policies WHERE schemaname='public' AND
        tablename='PolicyConfig' AND policyname='tenant_isolation_policy'
    ) THEN
        CREATE POLICY tenant_isolation_policy ON "PolicyConfig"
            USING ("tenant_id" = current_setting('app.tenant_id')::uuid);
    END IF;
END$$;

```

```
COMMIT;
```

Rollback — rollback/03_rls_disable_tenant.sql

```

BEGIN;
DROP POLICY IF EXISTS tenant_isolation_policy ON "AuditLog";
DROP POLICY IF EXISTS tenant_isolation_policy ON "TsaOutbox";
DROP POLICY IF EXISTS tenant_isolation_policy ON "PolicyConfig";

```

```
ALTER TABLE "AuditLog"      DISABLE ROW LEVEL SECURITY;
ALTER TABLE "TsaOutbox"     DISABLE ROW LEVEL SECURITY;
ALTER TABLE "PolicyConfig"  DISABLE ROW LEVEL SECURITY;
COMMIT;
```

4) Outbox com ON DELETE CASCADE — 04_outbox_ondetelete_cascade.sql

Mesma migração que já te entreguei, incorporada ao pacote (com pré-listagem de órfãos).

```
BEGIN;

WITH orphans AS (
    SELECT o.id
    FROM "TsaOutbox" o
    LEFT JOIN "AuditLog" a ON a.id = o."auditLogId"
    WHERE a.id IS NULL
)
SELECT COUNT(*) AS orphan_outbox FROM orphans;

DO $$
DECLARE fk_name text;
BEGIN
    SELECT conname INTO fk_name
    FROM pg_constraint c
    JOIN pg_class t ON t.oid = c.conrelid
    WHERE t.relname='TsaOutbox' AND c.contype='f'
    AND pg_get_constraintdef(c.oid) ILIKE '%"auditLogId"' REFERENCES
"AuditLog"%';
    IF fk_name IS NOT NULL THEN
        EXECUTE format('ALTER TABLE "TsaOutbox" DROP CONSTRAINT %I;', fk_name);
    END IF;
END$$;
```

```
CREATE INDEX IF NOT EXISTS "idx_tsaoutbox_auditlogid" ON "TsaOutbox"
("auditLogId");

ALTER TABLE "TsaOutbox"
    ADD CONSTRAINT "fk_tsaoutbox_auditlog_cascade"
    FOREIGN KEY ("auditLogId") REFERENCES "AuditLog"("id") ON DELETE CASCADE;

COMMIT;
```

Rollback — rollback/04_outbox_ondetelete_noaction.sql

```
BEGIN;

ALTER TABLE "TsaOutbox" DROP CONSTRAINT IF EXISTS
"fk_tsaoutbox_auditlog_cascade";

ALTER TABLE "TsaOutbox"
    ADD CONSTRAINT "fk_tsaoutbox_auditlog_noaction"
    FOREIGN KEY ("auditLogId") REFERENCES "AuditLog"("id") ON DELETE NO ACTION;

COMMIT;
```

Verificação pós-migração — verify.sql

```
-- 1) Enum Role ativo
SELECT t.typname, e.enumlabel
FROM pg_type t JOIN pg_enum e ON t.oid = e.enumtypid
WHERE t.typname='Role';

-- 2) Chave composta domain+version
SELECT conname, pg_get_constraintdef(oid) FROM pg_constraint
WHERE conrelid = 'PolicyConfig'::regclass AND contype='u';

-- 3) RLS habilitado
SELECT relname, relrowsecurity FROM pg_class
WHERE relname IN ('AuditLog','TsaOutbox','PolicyConfig');

-- 4) CASCADE na FK Outbox
SELECT conname, pg_get_constraintdef(oid) FROM pg_constraint
WHERE conrelid = 'TsaOutbox'::regclass AND contype='f';
```

Execução automatizada — scripts/run_release.sh

```
#!/usr/bin/env bash
set -euo pipefail

DB_URL="${DATABASE_URL:?missing}"
echo "== v2.0: preflight"
psql "$DB_URL" -f release/v2_0/preflight.sql

for f in 01_role_enum.sql 02_policy_domain_unique.sql 03_rls_enable_tenant.sql
04_outbox_ondetelete_cascade.sql; do
    echo "== applying $f"
    psql "$DB_URL" -f "release/v2_0/$f"
done

echo "== verify"
psql "$DB_URL" -f release/v2_0/verify.sql
```

Hashes & PoP-L — scripts/compute_hashes.sh

```
#!/usr/bin/env bash
set -euo pipefail
cd release/v2_0

for f in preflight.sql 01_role_enum.sql 02_policy_domain_unique.sql
03_rls_enable_tenant.sql 04_outbox_ondetelete_cascade.sql verify.sql; do
    shasum -a 512 "$f" | awk '{print $1, FILENAME}'
done
```

Evidência (preencher e registrar)

docs/EVIDENCE_PoP-L.json

```
{
  "release": "RLS-PoP-Enums-Outbox v2.0",
  "artifacts": [
    {"name": "01_role_enum.sql", "sha3_512": "<hex>", "tsa_token": "<base64>"},
    {"name": "02_policy_domain_unique.sql", "sha3_512": "<hex>", "tsa_token": "<base64>"}
  ],
}
```

```
{
  "name": "03_rls_enable_tenant.sql",
  "sha3_512": "<hex>",
  "tsa_token": "<base64>"
},
{
  "name": "04_outbox_ondetelete_cascade.sql",
  "sha3_512": "<hex>",
  "tsa_token": "<base64>"
}
],
"timestamp": "<ISO8601>",
"approved_by": "<userId_COMPLIANCE>",
"deployed_by": "<userId_ENGINEER>"
}
```

CI/CD (exemplo) — ci/github-actions.yml

```
name: release-v2-0
```

```
on: workflow_dispatch
```

```
jobs:
```

```
  dry-run:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - run: ./release/v2_0/scripts/compute_hashes.sh >
```

```
release/v2_0/docs/HASHES.txt
```

```
      - name: Preflight (staging)
```

```
        run: DATABASE_URL="${{ secrets.STAGING_DB }}" psql "$DATABASE_URL" -f
```

```
release/v2_0/preflight.sql
```

```
  apply-staging:
```

```
    needs: dry-run
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - name: Apply (staging)
```

```
        run: DATABASE_URL="${{ secrets.STAGING_DB }}"
```

```
./release/v2_0/scripts/run_release.sh
```

```
      - name: Gates pós-migração
```

```
        run: |
```

```
# Insira aqui consultas p95 e checagens SLO (FAST/DEEP/Outbox/TSA)
echo "SLO gates OK"
```

apply-prod:

needs: apply-staging

runs-on: ubuntu-latest

environment: production

steps:

- uses: actions/checkout@v4

- name: Apply (prod)

run: DATABASE_URL="\${{ secrets.PROD_DB }}"

./release/v2_0/scripts/run_release.sh

RELEASE_NOTES.md (resumo)

- **Role** → **ENUM** (evita drift e serializações livres).
- **PolicyConfig chave composta** (domain, version) (seleção inequívoca).
- **RLS habilitado** nas tabelas centrais; política tenant_isolation_policy.
- **Outbox CASCADE** para eliminar *dangling rows* e simplificar purgas/retention.
- **PoP-L**: hashes e TSA para todos os arquivos de migração e verificação.
- **Rollback**: scripts individuais por etapa.

Rollback orquestrado (se necessário)

1. Executar rollback/04_outbox_ondelete_noaction.sql
2. Executar rollback/03_rls_disable_tenant.sql
3. Executar rollback/02_policy_domain_unique_rollback.sql
4. Executar rollback/01_role_enum_rollback.sql

Ordem inversa preserva integridade de FKs/políticas.

Artefato #1 — código de produção do módulo Ω (Omega), o “motor de governança”. Ele recebe as métricas do MIR (r1.2) e as políticas configuradas, e decide se uma ação deve ser ALLOW, REVIEW ou BLOCK.

src/omega/omega.service.ts (r1.2 · FINAL)

```
import { Injectable, Logger } from '@nestjs/common';
import { CEL } from '@google-cloud/cel-common';
import type { MetricsInput } from '../mir/mir.service'; // MIR r1.2: contém aggregated_metrics
import type { PolicyConfig } from '@prisma/client';

/**
 * Esperado no PolicyConfig.rules (conforme dossiê):
 * [
 *   {
 *     name: string,
 *     priority: number,
 *     condition: string,      // expressão CEL usando prefixo "metrics."
 *     action: 'ALLOW'|'REVIEW'|'BLOCK',
 *     rationale?: string
 *   },
 *   ...
 * ]
 */
type DecisionAction = 'ALLOW' | 'REVIEW' | 'BLOCK';

interface DecisionPayload {
  action: DecisionAction;
  ruleTriggered: string | null;
  rationale: string | null;
}

@Injectable()
export class OmegaService {
```

```

private readonly logger = new Logger(OmegaService.name);
private readonly cel: CEL;

constructor() {
    this.cel = new CEL();
    this.logger.log('Motor de Governança (Omega) r1.2 inicializado.');
```

}

```

/**
 * (r1.2) Assinatura corrigida: agora recebe "MetricsInput" (MIR r1.2),
 * e avalia políticas no contexto { metrics:
metricsInput.aggregated_metrics }.
 */
public async govern(
    metricsInput: MetricsInput,
    policy: PolicyConfig,
): Promise<DecisionPayload> {
    const rules = Array.isArray((policy as any).rules) ? (policy as any).rules :
[];

    // Ordenação determinística (priority ASC, depois name ASC)
    const sortedRules = rules.slice().sort((a: any, b: any) => {
        if (a.priority !== b.priority) return a.priority - b.priority;
        return String(a.name).localeCompare(String(b.name));
    });

    // Binding r1.2: as regras acessam "metrics.<campo>"
    const env = {
        metrics: metricsInput.aggregated_metrics,
        // Se em algum momento quisermos expor claims/metadata de forma
controlada:
        // claims: metricsInput.claims,    // (opcional, conforme baseline Ω)
        // timestamp: metricsInput.timestamp
    };

```

```

for (const rule of sortedRules) {
  try {
    // Compila e avalia a expressão CEL
    const program = this.cel.createProgram(String(rule.condition));
    const result = await program.evaluate(env);

    if (result === true) {
      const action: DecisionAction = rule.action;
      const rationale = rule.rationale ?? null;

      this.logger.debug(`Ω regra acionada: [${rule.name}] → ${action}`);
      return {
        action,
        ruleTriggered: String(rule.name),
        rationale,
      };
    }
  } catch (err: any) {
    // Fail-safe (conforme dossiê): erro de regra → BLOCK
    this.logger.error(`Ω falha na regra "${rule?.name}": ${err?.message ??
err}`);
    return {
      action: 'BLOCK',
      ruleTriggered: 'FAIL_SAFE_RULE_ERROR',
      rationale: `Erro ao avaliar regra: ${String(rule?.name)}`,
    };
  }
}

// Nenhuma regra acionada
return { action: 'ALLOW', ruleTriggered: null, rationale: null };
}
}

```

Artefato #2 — arquivo de testes automatizados do Omega. Valida se o comportamento do artefato 1 está correto e estável.

src/omega/omega.service.spec.ts

(r1.2 · FINAL)

```
import { Test, TestingModule } from '@nestjs/testing';
import { OmegaService } from '../omega.service';
```

```
// Em projeto real, importar do MIR r1.2:
```

```
type MetricsInput = {
  aggregated_metrics: Record<string, any>;
  claims: any[];
  timestamp: string;
};
```

```
// Mock mínimo de PolicyConfig no formato esperado
```

```
type PolicyRule = {
  name: string;
  priority: number;
  condition: string; // CEL usando prefixo "metrics."
  action: 'ALLOW' | 'REVIEW' | 'BLOCK';
  rationale?: string;
};
```

```
type PolicyConfigMock = {
  id: string;
  rules: PolicyRule[];
};
```

```
const mockPolicy: PolicyConfigMock = {
  id: 'policy-omega-baseline',
  rules: [
    {
```

```

    name: 'BLOCK: PII Detectado',
    priority: 1,
    condition: 'metrics.pii_count > 0',
    action: 'BLOCK',
    rationale: 'PII não pode passar em produção',
  },
  {
    name: 'BLOCK: Risco Legal',
    priority: 5,
    condition: 'size(metrics.compliance_keywords_found) > 0',
    action: 'BLOCK',
    rationale: 'Termos de alto risco detectados',
  },
  {
    name: 'REVIEW: Sentimento Negativo',
    priority: 10,
    condition: 'metrics.plugin_sentiment_score < -0.7',
    action: 'REVIEW',
    rationale: 'Conteúdo potencialmente problemático',
  },
],
];
};

```

```

describe('OmegaService (r1.2)', () => {
  let service: OmegaService;

  beforeEach(async () => {
    const module: TestingModule = await Test.createTestingModule({
      providers: [OmegaService],
    }).compile();

    service = module.get<OmegaService>(OmegaService);
  });
});

```

```

it('deve decidir BLOCK se PII for detectado (prioridade 1)', async () => {
  const metrics: MetricsInput = {
    aggregated_metrics: { pii_count: 1, plugin_sentiment_score: 0.1,
compliance_keywords_found: [] },
    claims: [],
    timestamp: new Date().toISOString(),
  };
  const decision = await service.govern(metrics, mockPolicy as any);
  expect(decision.action).toBe('BLOCK');
  expect(decision.ruleTriggered).toBe('BLOCK: PII Detectado');
});

it('deve decidir BLOCK se houver risco legal (prioridade 5)', async () => {
  const metrics: MetricsInput = {
    aggregated_metrics: { pii_count: 0, plugin_sentiment_score: 0.2,
compliance_keywords_found: ['garantia'] },
    claims: [],
    timestamp: new Date().toISOString(),
  };
  const decision = await service.govern(metrics, mockPolicy as any);
  expect(decision.action).toBe('BLOCK');
  expect(decision.ruleTriggered).toBe('BLOCK: Risco Legal');
});

it('deve decidir REVIEW se o sentimento for muito negativo (prioridade 10)',
async () => {
  const metrics: MetricsInput = {
    aggregated_metrics: { pii_count: 0, plugin_sentiment_score: -0.9,
compliance_keywords_found: [] },
    claims: [],
    timestamp: new Date().toISOString(),
  };
  const decision = await service.govern(metrics, mockPolicy as any);
  expect(decision.action).toBe('REVIEW');
  expect(decision.ruleTriggered).toBe('REVIEW: Sentimento Negativo');
});

```

```

it('deve decidir ALLOW se nenhuma regra for acionada', async () => {
  const metrics: MetricsInput = {
    aggregated_metrics: { pii_count: 0, plugin_sentiment_score: 0.5,
compliance_keywords_found: [] },
    claims: [],
    timestamp: new Date().toISOString(),
  };
  const decision = await service.govern(metrics, mockPolicy as any);
  expect(decision.action).toBe('ALLOW');
  expect(decision.ruleTriggered).toBeNull();
});

it('fail-safe: deve bloquear quando a expressão CEL é inválida', async () => {
  const badPolicy: PolicyConfigMock = {
    id: 'policy-bad',
    rules: [
      {
        name: 'Regra inválida',
        priority: 1,
        condition: 'metrics.pii_count >', // sintaxe propositalmente quebrada
        action: 'ALLOW',
      },
    ],
  };
  const metrics: MetricsInput = {
    aggregated_metrics: { pii_count: 0 },
    claims: [],
    timestamp: new Date().toISOString(),
  };
  const decision = await service.govern(metrics, badPolicy as any);
  expect(decision.action).toBe('BLOCK');
  expect(decision.ruleTriggered).toBe('FAIL_SAFE_RULE_ERROR');
});

```

```

it('prioridade: regra de menor prioridade não deve sobrepor uma de maior
prioridade', async () => {
  const policy: PolicyConfigMock = {
    id: 'policy-priority',
    rules: [
      { name: 'REVIEW baixo', priority: 50, condition:
'metrics.plugin_sentiment_score < -0.2', action: 'REVIEW' },
      { name: 'BLOCK crítico', priority: 1, condition: 'metrics.pii_count >
0', action: 'BLOCK' },
    ],
  };
  const metrics: MetricsInput = {
    aggregated_metrics: { pii_count: 1, plugin_sentiment_score: -0.9 },
    claims: [],
    timestamp: new Date().toISOString(),
  };
  const decision = await service.govern(metrics, policy as any);
  expect(decision.action).toBe('BLOCK');
  expect(decision.ruleTriggered).toBe('BLOCK crítico');
});
});

```

Checklist de PR (curto e objetivo)

- **Assinatura r1.2 aplicada:** `govern(metricsInput: MetricsInput, policy: PolicyConfig)`
 - **Binding correto** no CEL: `metrics: metricsInput.aggregated_metrics`
 - **Regras existentes** continuam do tipo `metrics.<campo>` (não “quebramos” políticas)
 - **Testes r1.2** passam cobrindo: PII, risco legal, sentimento, nenhum match, **erro de regra, prioridade**
 - **Lint de políticas Ω** (P2.36) sem violações
 - Relatório V&V inclui `model_id`, `seed`, `commit`, `environment_hash` (P1.33)
 - **Apêndice SLOs** presente no PDF/A (P2.34) e evidência PoP-L (hash + TSA) anexada
 - **OpenAPI 1.0** intacto (P1.45) e FSM/RBR consistentes
-

Comandos úteis (exemplos)

testes unitários

```
npm run test -- src/omega/omega.service.spec.ts
```

lint (código + políticas Ω , se exposto via script)

```
npm run lint
```

```
npm run policies:lint
```

V&V mínimo (onde já configurado)

```
npm run vv:run && npm run vv:report
```
